UNIVERSITY OF ZAMBIA

CSC 6302 — Computer Vision

Practical 1 Report: Seeing Like a Machine

Student: Munyeme Sinyani (SINYANI)
Computer number: 2025024319
Department: Computing and Informatics
Photos: colourful.jpg, lowlight.jpg, coins.jpg (earrings)

Task 1 — Colour spaces & colour picker

I loaded my outdoor colourful scene and converted it to HSV and LAB. In HSV, **H** is the colour type (angle on the colour wheel), **S** is how pure the colour is, and **V** is brightness. In LAB, **L** is lightness, **A** runs green↔red, and **B** runs blue↔yellow. I picked the lime-green motorcycle with `cv2.inRange` on HSV. This is easier than RGB because hue stays fairly stable when sunlight changes the brightness; in RGB the same green would spread across all three channels as lighting changes.

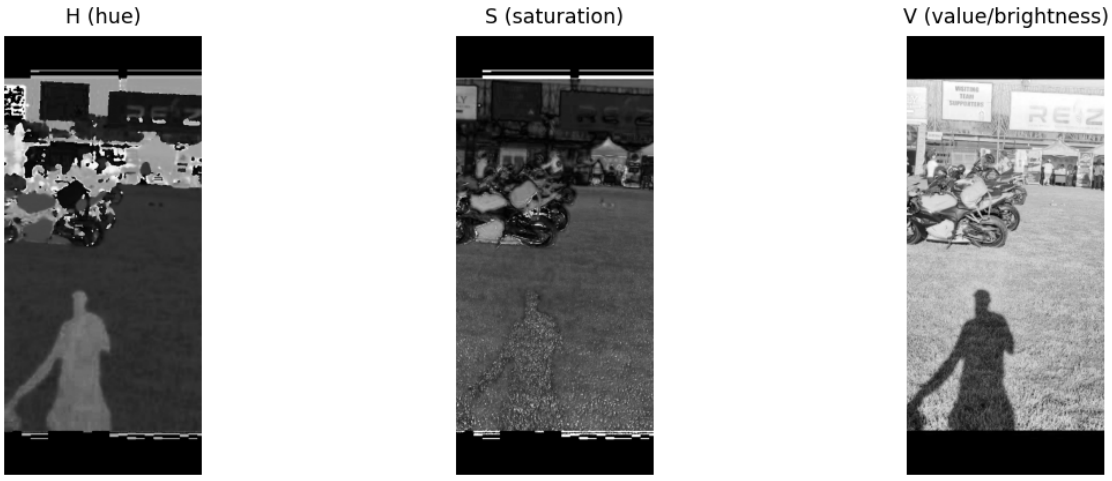


Figure 1. HSV channels (H, S, V).

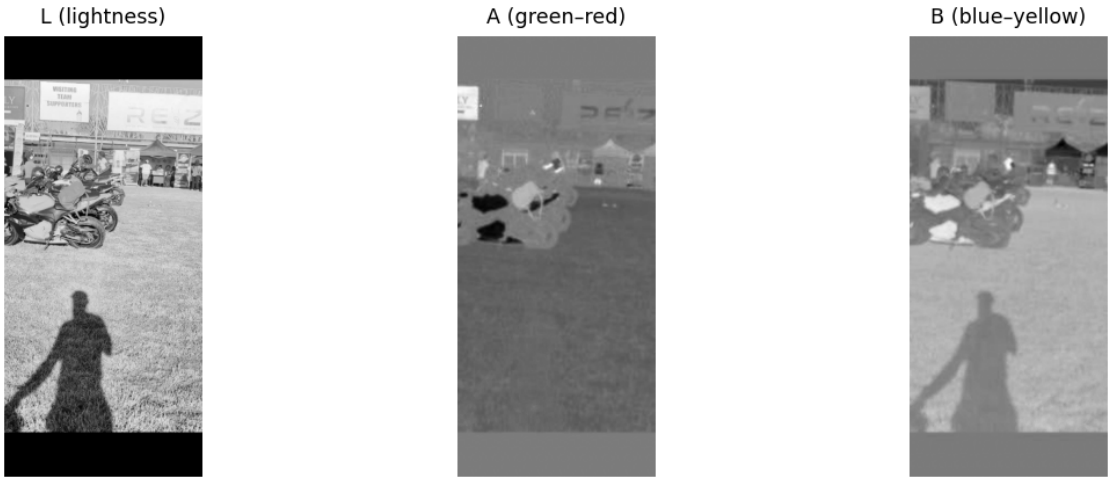


Figure 2. LAB channels (L, A, B).

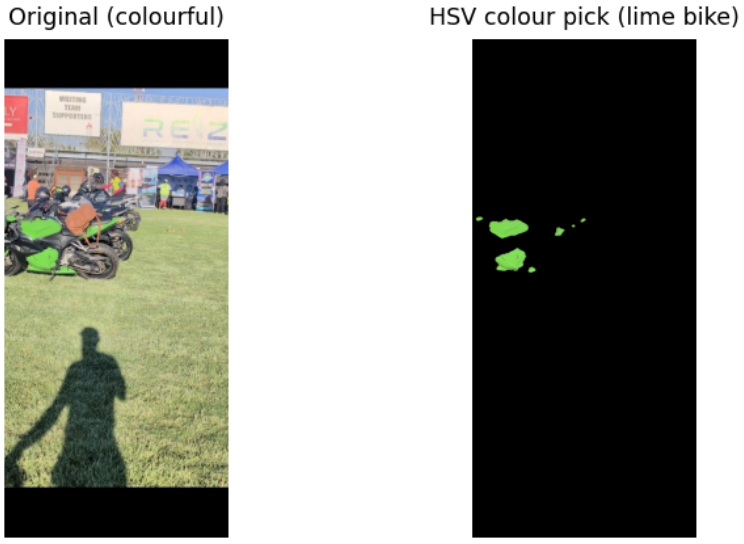


Figure 3. HSV colour picker isolating the lime-green bike.

Low-light / noisy input photo



Figure 4. My low-light / noisy input photo (setup requirement).

Task 2 — Noise and smoothing (PSNR from scratch)

From a clean grayscale of my colourful scene I synthesised Gaussian noise (σ=15) and 5% salt-and-pepper noise. I restored each with box, Gaussian and median filters at kernel sizes 3, 5 and 7. PSNR is implemented in NumPy as 10·log■■(MAX²/MSE) with MAX=255 — no `cv2.PSNR`.

Noise	Filter	k	PSNR (dB)
gaussian	box	3	23.696
gaussian	gaussian	3	25.152
gaussian	median	3	23.716
gaussian	box	5	22.317
gaussian	gaussian	5	24.012
gaussian	median	5	22.653
gaussian	box	7	21.546
gaussian	gaussian	7	23.061
gaussian	median	7	21.988
sap	box	3	22.194
sap	gaussian	3	22.775
sap	median	3	24.175
sap	box	5	21.784
sap	gaussian	5	22.781
sap	median	5	22.8
sap	box	7	21.25
sap	gaussian	7	22.4
sap	median	7	22.063

Winner: for Gaussian noise the Gaussian filter at k=3 has the highest PSNR (25.15 dB). For salt-and-pepper, median at k=3 wins (24.18 dB). That matches the lecture idea: mean/Gaussian filters blend outliers into neighbours, while the median can throw salt/pepper spikes away completely.



Figure 5. Clean vs Gaussian vs salt-and-pepper.

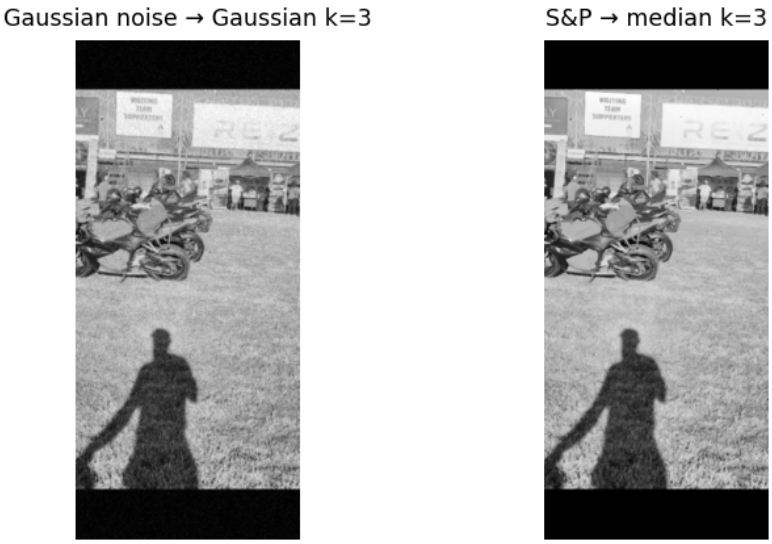


Figure 6. Best restorations for each noise type.

Task 3 — Unsharp masking

I built sharpening as `sharpened = original + amount × (original − blurred)` in float32, then clipped back to uint8. Amounts 0.5, 1.0 and 2.0 are shown. At amount=2.0 bright/dark rings (halos) appear around strong edges — the high-pass residual is too strong.

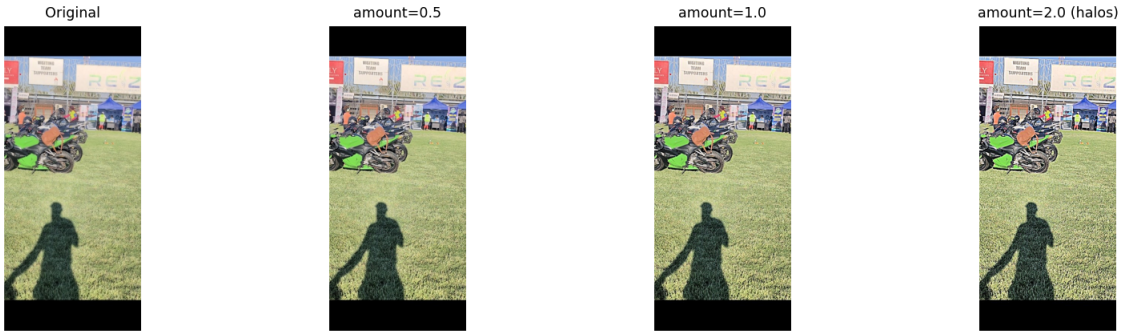


Figure 7. Unsharp masking for amount = 0.5, 1.0, 2.0.

Task 4 — Edge detection three ways

I implemented 2-D convolution with nested loops and Sobel kernels, then compared the magnitude to `cv2.Sobel`. Result: **max_abs_diff=0.000000e+00** after using reflect padding (OpenCV's default border). Canny uses hysteresis: the high threshold marks strong edges; the low threshold keeps weak edges only if they connect to strong ones. LoG (blur then Laplacian) responds to zero-crossings and looks more "blob/outline" like, while Sobel emphasises directed gradient strength.

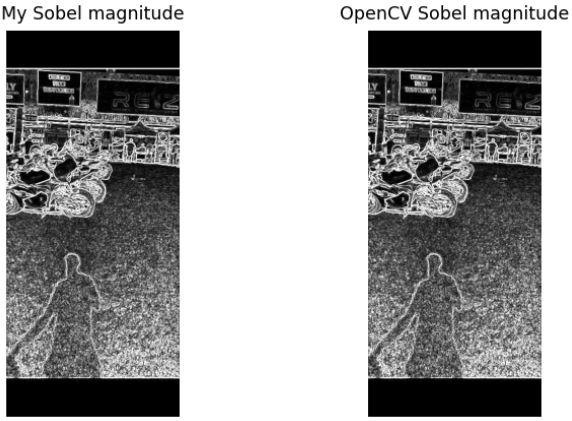


Figure 8. My Sobel vs OpenCV Sobel.

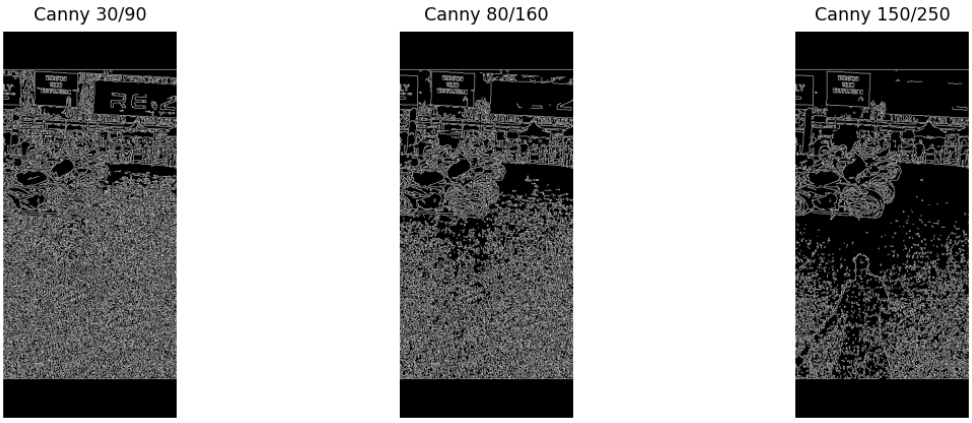


Figure 9. Canny at three threshold pairs.

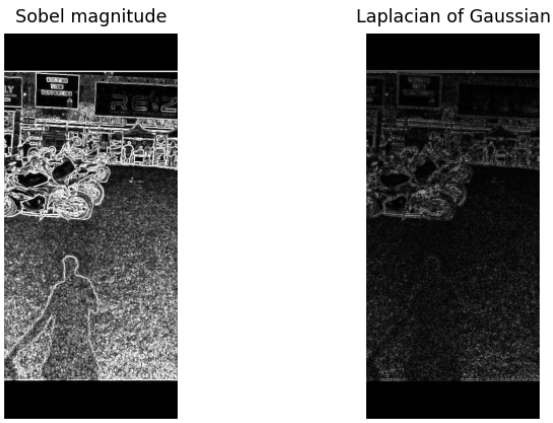


Figure 10. Sobel magnitude vs Laplacian of Gaussian.

Task 5 — Morphology & object counting

My countable photo is fashion earrings on a white card (`coins.jpg`). I cropped to the jewellery band (to avoid counting the printed letters), used Otsu thresholding (inverted so metal is white), then erosion, dilation, opening and closing with a **5×5 ellipse** structuring element — small enough to keep studs separate, large enough to fill small holes in hoop outlines. Finally `connectedComponentsWithStats` with an area gate. Pipeline result: **object_count=15**. By eye I count about 18 earring pieces on the card; the automatic count is lower because some neighbouring pieces merge after closing.



Figure 11. Threshold → erode → dilate → open → close.



Figure 12. Input photo and counted bounding boxes.

Task 6 — CLI toolbox & reflection

I wrapped the work into `prac1/process.py` with operations `pick | denoise | sharpen | edges | count` (five ops; brief asks for at least four). Example: `python prac1/process.py --input images/coins.jpg --op count --output out.png`.

Reflection (failure → lecture concept). The place my pipeline fails most clearly is object counting when earrings touch or when reflections split one earring into several white blobs. Closing can glue two studs into one component; opening can erase a thin hoop fragment. That is exactly the morphology trade-off from lectures: structuring-element size encodes a prior about object scale. A second failure mode is the colour picker under mixed lighting — grass and the lime bike share greenish hues, so saturation thresholds matter. Both failures are about the gap between “pixels” and “objects”: filtering and morphology are local number tricks, not full scene understanding. Later practicals (features / learning) try to close that gap.

Reproducibility & AI declaration

Commands are in `prac1/README.md` with `prac1/requirements.txt`. Run `python prac1/run_prac1.py` to regenerate every figure. Composer helped with script layout; from-scratch PSNR/Sobel and this reflection I can explain line by line.